WG Finanztracker

Projektbericht zur Hausarbeit

31. August 2026

Inhaltsverzeichnis

1 Einleitung **3**
- 1.1 Motivation . . . 3
- 1.2 Zielsetzung . . . 3

2 Theoretische Grundlagen und Werkzeuge **3**
- 2.1 Technologien . . . 3
- 2.2 OCR und Bildverarbeitung . . . 4
- 2.3 Entwicklung und Evaluation der OCR-Lösung . . . 4
- 2.4 Statische Codeanalyse und Tests . . . 5
- 2.5 Mensch vs. KI: Die Rolle von AI Assistance . . . 5

3 Konzept und Implementierung **6**
- 3.1 Programmstruktur . . . 6
- 3.2 Datenmodell . . . 7
- 3.3 Kostenaufteilung . . . 7
- 3.4 Zahlungen . . . 7
- 3.5 OCR-Pipeline . . . 8
- 3.6 Parsing der Kassenboninformationen . . . 8
- 3.7 Benutzerbearbeitung nach der OCR . . . 8
- 3.8 Fehlerbehandlung . . . 9
- 3.9 Tests und Qualitätssicherung . . . 9
- 3.10 Design- und Technologieentscheidungen . . . 9

4 Ergebnisse **10**
- 4.1 Funktionsnachweis . . . 10
- 4.2 Beispiel-Durchlauf . . . 10
- 4.3 Ergebnisse visualisieren . . . 11

5 Diskussion und Fazit **12**
- 5.1 Zielerreichung . . . 12
- 5.2 Herausforderungen . . . 12
- 5.3 Kritische Reflexion . . . 12
- 5.4 Abweichungen vom ursprünglichen Plan . . . 13

6 Ausblick **13**

7 Formale Qualitätssicherung **13**

8 Design by Contract und verwandte Techniken **13**

9 Fazit **13**

A Ausgewählter Code-Snippet **14**
A.1 Bewertung von OCR-Ergebnissen 14

1 Einleitung

1.1 Motivation

Bei gemeinschaftlichen Einkäufen in Wohngemeinschaften (WG) entsteht regelmäßig das Problem, dass eine Person den gesamten Einkauf bezahlt, die enthaltenen Artikel jedoch von mehreren Personen genutzt werden. Eine einfache Aufteilung des Gesamtbetrags auf alle Mitglieder ist dabei häufig nicht sinnvoll, da einzelne Produkte nur von bestimmten Personen gekauft werden.

Das Ziel des Projekts war deshalb die Entwicklung eines webbasierten WG-Finanztrackers, mit dem Kassenbons erfasst, deren Artikel verwaltet und einzelnen Mitgliedern zugeordnet werden können. Auf Grundlage dieser Zuordnungen berechnet die Anwendung die jeweiligen Anteile und daraus resultierende offene Beträge.

Ein besonderer Schwerpunkt lag auf der automatisierten Erfassung von Kassenbons. Hierzu wurde eine OCR-Pipeline auf Basis von Tesseract und OpenCV entwickelt. Da Kassenbons je nach Händler, Druckqualität, Perspektive und Layout unterschiedlich aufgebaut sind, wurde die Erkennung nicht auf einen einzelnen Händler zugeschnitten, sondern möglichst allgemein konzipiert.

1.2 Zielsetzung

Die Zielsetzung orientierte sich an der Projektskizze. Die Anwendung sollte insbesondere folgende Funktionen bereitstellen:

- Verwaltung von WG-Mitgliedern,
- Erfassung und Verwaltung von Kassenbons,
- automatisierte OCR-Erkennung von Kassenbons,
- Bearbeitung erkannter Daten vor dem Speichern,
- Zuordnung von Artikeln zu einem oder mehreren Mitgliedern,
- Berechnung individueller Kostenanteile,
- Berechnung offener Schulden zwischen Mitgliedern,
- Erfassung bereits geleisteter Zahlungen,
- globale Übersicht über offene Beträge,
- statistische Auswertungen der Ausgaben.

Die OCR-Erkennung wurde dabei bewusst als unterstützende Funktion umgesetzt. Aufgrund der unterschiedlichen Gestaltung von Kassenbons werden erkannte Werte vor dem endgültigen Speichern weiterhin durch den Benutzer überprüft und können korrigiert werden.

2 Theoretische Grundlagen und Werkzeuge

2.1 Technologien

Das Projekt wurde als Python-Webanwendung mit Flask umgesetzt. Für die Persistenz der Daten kommt PostgreSQL zum Einsatz. Die Abhängigkeiten und grundlegenden Projektinformationen werden in der `pyproject.toml` verwaltet.

Die wesentlichen verwendeten Technologien und Bibliotheken sind:

- **Python** – Programmiersprache
- **Flask, Jinja2** – Webanwendung, Routing und Templates
- **PostgreSQL, psycopg** – Persistenz und Datenbankzugriff
- **Bootstrap** – Gestaltung der Weboberfläche
- **Tesseract OCR, pytesseract** – Texterkennung
- **OpenCV, NumPy** – Bildverarbeitung und Vorverarbeitung

- **pytest, Hypothesis** – automatisierte Tests und Property-Based Testing
- **mypy** – statische Typprüfung
- **Ruff** – Linting und Formatierung
- **Interrogate** – Überprüfung der Dokumentation

2.2 OCR und Bildverarbeitung

Für die Kassenbonerkennung wird zunächst das Eingabebild verarbeitet. Die Pipeline verwendet unter anderem Graustufen, Skalierung, Rauschunterdrückung, Kontrastverbesserung und perspektivische Korrektur. Ein wesentlicher Bestandteil ist die Erzeugung mehrerer Bildvarianten. Neben einer kontrastverbesserten Variante werden unter anderem eine beleuchtungsnormalisierte, eine Otsu-binarisierte und eine adaptiv binarisierte Variante erzeugt. Zusätzlich wird versucht, den Kassenbon im ursprünglichen Foto zu erkennen und perspektivisch zu entzerren. Dadurch soll verhindert werden, dass Hintergrundflächen oder eine starke Schräglage die Texterkennung unnötig erschweren. Auf jeder relevanten Bildvariante wird Tesseract mit mehreren Page-Segmentation-Modi ausgeführt. Die resultierenden Kandidaten werden anschließend anhand verschiedener Kriterien bewertet.

2.3 Entwicklung und Evaluation der OCR-Lösung

Die OCR-Komponente wurde im Verlauf des Projekts mehrfach überarbeitet. Dabei wurde nicht von Beginn an die letztendlich verwendete Lösung vorgegeben, sondern verschiedene OCR-Bibliotheken und Ansätze praktisch erprobt.

Der erste Ansatz basierte auf einer direkten Verwendung von Tesseract OCR. Tesseract wurde zunächst gewählt, da es sich um eine etablierte Open-Source-OCR-Lösung handelt und eine Python-Schnittstelle mit pytesseract zur Verfügung steht. Dadurch ließ sich die Texterkennung vergleichsweise einfach in die bestehende Python-Anwendung integrieren. Hierbei wurde das Eingabebild vorverarbeitet und anschließend mit einer einzelnen Tesseract-Konfiguration ausgewertet. Die ersten Tests zeigten jedoch, dass die Ergebnisse bei unterschiedlichen Kassenbonfotos stark schwanken konnten. Insbesondere wurden Artikel, Preise und Metadaten teilweise miteinander verwechselt. Aufgrund dieser Ergebnisse wurde zunächst EasyOCR untersucht und in die Anwendung integriert. Dabei wurde EasyOCR als Alternative betrachtet, da es eine einfache Python-Schnittstelle bietet und einen anderen OCR-Ansatz als Tesseract verwendet. Ziel war es zu überprüfen, ob sich die Erkennungsqualität bei den im Projekt verwendeten Kassenbonfotos durch einen Wechsel der OCR-Bibliothek verbessern lässt. Die erzielten Ergebnisse führten jedoch nicht zu einer ausreichenden Verbesserung für die im Projekt betrachteten Kassenbons. Anschließend wurde PaddleOCR untersucht und ebenfalls praktisch getestet. Die Bibliothek wurde ausgewählt, da sie speziell für die Erkennung von Text in Bildern ausgelegt ist und eine weitere Alternative zu den bisher getesteten Ansätzen darstellt. Dabei zeigte sich jedoch, dass auch mit PaddleOCR das grundlegende Problem der unterschiedlichen Kassenbonlayouts nicht vollständig gelöst werden konnte. Die Schwierigkeiten lagen somit nicht ausschließlich in der verwendeten OCR-Bibliothek, sondern auch in der Vorverarbeitung, der Textinterpretation und der anschließenden Zuordnung von Artikeln und Preisen.

Aus diesem Grund wurde schließlich erneut Tesseract verwendet. Im Unterschied zur ursprünglichen Implementierung wurde die Pipeline jedoch deutlich erweitert. Statt einer einzelnen OCR-Auswertung werden nun mehrere Bildvarianten erzeugt und mit unterschiedlichen Page-Segmentation-Modi verarbeitet. Die verschiedenen Ergebnisse werden anschließend als OCR-Kandidaten bewertet und miteinander verglichen.

Die letztendliche Entscheidung für Tesseract war daher keine reine Auswahl nach Bibliotheksumfang, sondern das Ergebnis mehrerer Implementierungs- und Testschritte. Der Schwerpunkt verlagerte sich von der Frage nach der vermeintlich besten OCR-Bibliothek hin zur Frage, wie

die Eingabebilder und die resultierenden OCR-Daten so verarbeitet werden können, dass auch unterschiedliche Kassenbonlayouts berücksichtigt werden.

Diese Entwicklung war insbesondere deshalb relevant, weil die Anforderung nicht auf einen einzelnen Händler beschränkt war. Eine Lösung, die ausschließlich für beispielsweise Lidl oder ALDI gut funktioniert, wäre für das eigentliche Ziel des Projekts nicht ausreichend gewesen.

2.4 Statische Codeanalyse und Tests

Für die formale Qualitätssicherung wurden mehrere Werkzeuge eingesetzt. Mypy prüft die im Projekt verwendeten Type-Hints. Ruff übernimmt sowohl die Prüfung des Quelltexts als auch die Formatierung. Mit Interrogate wird der Anteil dokumentierter Funktionen überprüft.

Für die automatisierten Tests kommt pytest zum Einsatz. Zusätzlich werden Hypothesis-Tests verwendet, um Eigenschaften der Kostenaufteilung mit einer größeren Anzahl automatisch erzeugter Eingaben zu überprüfen.

2.5 Mensch vs. KI: Die Rolle von AI Assistance

Bei der Entwicklung wurde ChatGPT als unterstützendes Werkzeug eingesetzt. Die Nutzung erfolgte dabei nicht ausschließlich in Form einzelner Frage-Antwort- Interaktionen. Vielmehr wurde ChatGPT über den Entwicklungsprozess hinweg für intensive, iterative Diskussionen eingesetzt. Dabei wurden beispielsweise Entwurfsentscheidungen hinterfragt, verschiedene Lösungsansätze miteinander verglichen, Implementierungen diskutiert und anschließend anhand der tatsächlichen Ergebnisse weiter angepasst. Der Einsatz erfolgte insbesondere bei der Diskussion von Architekturentscheidungen, der Analyse von Fehlermeldungen, der Entwicklung, sowie bei Refactoring und Dokumentation.

Ein konkretes Beispiel war die Weiterentwicklung der OCR-Pipeline. Die grundlegende Idee, mehrere OCR-Varianten parallel auszuwerten und anschließend geeignete Ergebnisse auszuwählen beziehungsweise zu einem belastbareren Gesamtergebnis zu kombinieren, wurde selbst entwickelt. Die konkrete Umsetzung wurde anschließend in mehreren Iterationen mit ChatGPT diskutiert. Dabei wurden mögliche Verfahren, deren Vor- und Nachteile sowie Probleme der jeweiligen Implementierung besprochen. Die daraus entstandenen Vorschläge wurden implementiert und anhand der tatsächlichen Ergebnisse überprüft. Auch bei der Auswahl und Anpassung der OCR-Verfahren wurde KI-Unterstützung genutzt. Die finale Entscheidung für die verwendete Tesseract-Pipeline resultierte somit aus eigenen Tests und den daraus geführten Diskussionen und nicht allein aus einer KI-Empfehlung. Für die Weboberfläche wurden teilweise HTML-Templates aus einem früheren, selbst entwickelten Projekt wiederverwendet. Dies diente dazu, bereits vorhandene und erprobte Grundstrukturen nicht erneut entwickeln zu müssen und den Entwicklungsaufwand auf die projektspezifischen Funktionen zu konzentrieren. Ebenfalls wurden Beispieldaten für die Datenbank mithilfe KI generiert. Die bestehenden Templates wurden anschließend mit Unterstützung von ChatGPT an die Anforderungen und das gewünschte Erscheinungsbild des aktuellen Projekts angepasst. Auch beim Refactoring wurde KI-Unterstützung verwendet. Mypy- und Ruff-Meldungen wurden analysiert und mögliche Lösungen diskutiert. Die vorgeschlagenen Änderungen wurden anschließend anhand des tatsächlich vorhandenen Codes überprüft und bei Bedarf manuell angepasst. Die eigentliche Validierung erfolgte durch das Ausführen der Anwendung, automatisierte Tests sowie die statischen Prüfungen mit mypy und Ruff.

ChatGPT wurde darüber hinaus bei der Erstellung dieses Projektberichts eingesetzt. Dazu gehörten insbesondere die Strukturierung einzelner Abschnitte, sprachliche Überarbeitungen sowie die Diskussion der Darstellung technischer Entscheidungen und Ergebnisse. Die fachlichen Inhalte, die Beschreibung der tatsächlichen Implementierung und die Auswahl der dargestellten Ergebnisse wurden selbst überprüft und an den tatsächlichen Projektstand angepasst.

KI wurde damit als Entwicklungsassistenz eingesetzt und nicht als Ersatz für die eigene Konzeption, Implementierung und Überprüfung der Lösung. Die konkrete Nutzung von KI wurde entsprechend den Vorgaben in der Eigenständigkeitserklärung berücksichtigt.

3 Konzept und Implementierung

3.1 Programmstruktur

Die Anwendung ist in mehrere Bereiche unterteilt. Die Domain enthält die fachliche Logik und Datenmodelle. Die Persistence-Schicht kapselt den Datenbankzugriff. Der OCR-Bereich enthält die Bildverarbeitung, Texterkennung und das Parsing. Die Web-Schicht stellt diese Funktionen über Flask zur Verfügung.

Eine vereinfachte Struktur des Projekts ist:

```
src/
├── wg_finanztracker/
│   ├── domain/
│   │   ├── models.py
│   │   ├── cost_sharing.py
│   │   └── stats.py
│   ├── ocr/
│   │   ├── cli.py
│   │   ├── ocr_models.py
│   │   ├── preprocessing.py
│   │   ├── quality.py
│   │   ├── receipt_parser.py
│   │   ├── service.py
│   │   └── tesseract_service.py
│   ├── persistence/
│   │   └── database.py
│   ├── web/
│   │   └── app.py
│   └── resources/
│       ├── schema.sql
│       ├── data.sql
│       └── templates/
│           ├── 404.html
│           ├── index.html
│           ├── members.html
│           ├── member_edit.html
│           ├── receipts.html
│           ├── receipt_detail.html
│           ├── receipt_edit.html
│           ├── receipt_upload.html
│           ├── statistics.html
│           └── template.html
│       └── static/
│           └── css/
│               └── bootstrap.min.css
└── tests/
    └── ...
```

Die Trennung dient insbesondere dazu, die fachliche Berechnung von Datenbank- und Weblogik zu entkoppeln. Dadurch können beispielsweise Funktionen zur Kostenaufteilung getestet

werden, ohne die Flask-Anwendung starten zu müssen.

3.2 Datenmodell

Das zentrale fachliche Modell besteht aus den Entitäten `Member`, `Receipt`, `Item` und `Payment`.

Ein `Receipt` enthält Händler, Kaufdatum, Gesamtbetrag, die Person, die den Einkauf bezahlt hat, sowie die zugehörigen Artikel. Ein `Item` besitzt einen Namen, einen Preis und eine Liste zugeordneter Mitglieder. Dadurch können Artikel sowohl einer einzelnen Person als auch mehreren Personen zugeordnet werden. Ein `Payment` repräsentiert dagegen eine bereits erfolgte Zahlung zwischen zwei Mitgliedern. Das Modell enthält Absender, Empfänger, Betrag und Datum. Auf Datenbankebene werden diese Beziehungen unter anderem durch die Tabellen `members`, `receipts`, `items`, `item_assignments` und `payments` abgebildet.

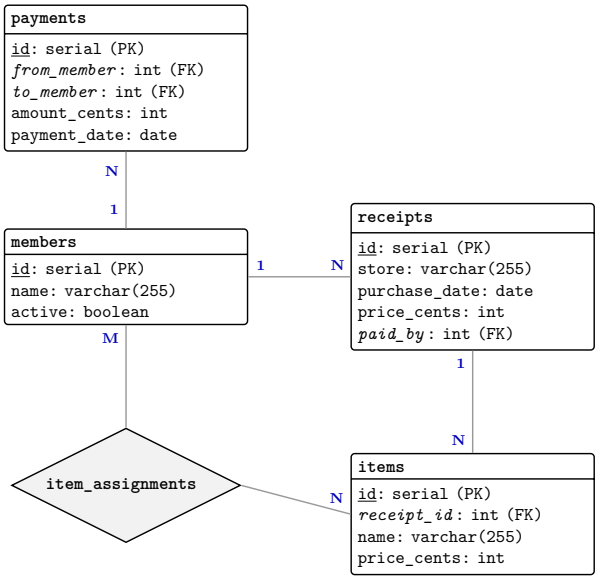


Abbildung 1: ER-Modell mit Kardinalitäten

3.3 Kostenaufteilung

Die Kostenaufteilung erfolgt auf Artikelebene. Für jeden Artikel wird dessen Preis gleichmäßig auf die ihm zugeordneten Personen verteilt. Geldbeträge werden ausschließlich in Cent gespeichert, da sich dadurch Rundungsfehler bei Berechnungen mit ganzen Geldbeträgen vermeiden lassen. Eine Division kann zwar weiterhin einen Rest ergeben, dieser wird jedoch deterministisch auf die ersten Mitglieder der Zuordnung verteilt. Für einen Artikel mit einem Preis von beispielsweise 100 Cent und vier zugeordneten Personen ergeben sich damit Anteile von 25, 25, 25 und 25 Cent. Die Summe der Anteile bleibt dabei immer exakt gleich dem ursprünglichen Artikelpreis. Aus den individuellen Anteilen wird anschließend bestimmt, welche Mitglieder dem jeweiligen Bezahler Geld schulden.

Für mehrere Kassenbons werden diese Beträge zu einer globalen Schuldensituation zusammengeführt. Bereits erfasste `Payment`-Objekte werden dabei berücksichtigt und reduzieren die offenen Beträge.

3.4 Zahlungen

Zur Abbildung bereits ausgeglichener Schulden wurde das Modell `Payment` eingeführt. Ein Payment beschreibt eine tatsächliche Geldbewegung, beispielsweise: *Max bezahlt Deniz 5 Euro*. Bei der Berechnung der globalen Schulden werden zunächst die offenen Beträge aus allen Kassenbons berechnet. Anschließend werden die gespeicherten Zahlungen berücksichtigt.

Dadurch kann beispielsweise eine ursprüngliche Schuld von 10 Euro durch eine Zahlung von 4 Euro auf 6 Euro reduziert werden. Eine vollständige Zahlung führt dazu, dass der entsprechende offene Betrag aus der Übersicht verschwindet.

3.5 OCR-Pipeline

Die im vorherigen Abschnitt beschriebene Entwicklung führte zur letztendlich verwendeten Tesseract-Pipeline. Diese besteht aus mehreren aufeinanderfolgenden Verarbeitungsschritten und ist bewusst nicht auf das Layout eines einzelnen Händlers zugeschnitten.

1. Laden des Bildes,
2. Erkennung und perspektivische Korrektur des Kassenbonbereichs,
3. Skalierung und Graustufenumwandlung,
4. Rauschunterdrückung,
5. Verbesserung der Bildqualität,
6. Erzeugung mehrerer Bildvarianten,
7. OCR mit mehreren Tesseract-Konfigurationen,
8. Parsing der erkannten Texte,
9. Bewertung der einzelnen OCR-Kandidaten,
10. Kombination mehrerer Kandidaten zu einem Ergebnis.

Die Nutzung mehrerer Varianten wurde gewählt, weil keine einzelne Vorverarbeitung für alle Kassenbonfotos optimal ist. Ein Bild kann beispielsweise bei gleichmäßiger Beleuchtung mit einer Graustufenvariante besser funktionieren, während bei Schatten oder ungleichmäßiger Beleuchtung eine normalisierte Variante Vorteile bietet.

Die Kandidatenbewertung berücksichtigt unter anderem das Vorhandensein eines Händlers, eines Datums und eines Gesamtbetrags sowie die Plausibilität der erkannten Artikel. Dadurch soll verhindert werden, dass ausschließlich die von Tesseract gemeldete Zeichensicherheit über das endgültige Ergebnis entscheidet.

3.6 Parsing der Kassenboninformationen

Das Parsing verwendet reguläre Ausdrücke und heuristische Regeln. Preise werden aus erkannten Zahlenmustern extrahiert und direkt in Cent umgerechnet. Für das Erkennen des Gesamtbetrags werden typische Begriffe wie `Summe`, `Gesamt` oder `Zu zahlen` berücksichtigt. Zusätzlich werden Steuerzeilen als unabhängige Plausibilisierung verwendet, wenn entsprechende Informationen auf dem Bon vorhanden sind. Bei Händlern werden einige bekannte Namen direkt erkannt. Gleichzeitig existiert eine allgemeine heuristische Erkennung für unbekannte Händler, sodass die Implementierung nicht ausschließlich auf eine feste Händlerliste angewiesen ist. Artikel werden anhand ihrer geometrischen Position und ihrer Preisposition erkannt. Dabei wird versucht, die typische Preis-Spalte eines Kassenbons zu bestimmen und Metadaten wie Steuer-, Zahlungs- oder Rabattzeilen auszuschließen.

3.7 Benutzerbearbeitung nach der OCR

Die OCR-Erkennung liefert kein endgültig gespeichertes Datenobjekt. Stattdessen werden die erkannten Werte in einem Bearbeitungsformular angezeigt. Händler, Datum, Gesamtbetrag und Artikel können vor dem Speichern kontrolliert und korrigiert werden. Zusätzlich kann jedem Artikel ein oder mehreren WG-Mitgliedern zugeordnet werden. Diese Vorgehensweise wurde gewählt, weil OCR-Fehler bei Fotos von Kassenbons nicht vollständig vermieden werden können. Die Anwendung verwendet OCR deshalb als Beschleunigung der Eingabe und nicht als alleinige Datenquelle.

3.8 Fehlerbehandlung

Fehlerhafte oder unvollständige Eingaben werden an verschiedenen Stellen abgefangen. Beispiele sind:

- fehlende Datei beim Upload,
- nicht unterstützte Dateiformate,
- ungültige Preisangaben/Datumswerte,
- nicht existierende Mitglieder,
- fehlende Artikelzuordnung,
- nicht vorhandene Datensätze,
- unbekannte URL-Pfade.

Für nicht vorhandene Seiten wurde ein eigener HTTP-404-Handler implementiert. Ungültige Formulareingaben führen zu einer Fehlermeldung auf der bestehenden Eingabeseite und nicht zu einem unbehandelten Programmabbruch. Auch in der Domainlogik werden ungültige Zustände explizit behandelt. Ein Artikel ohne zugeordnete Mitglieder führt beispielsweise zu einem `ValueError`, anstatt später während der Division einen weniger aussagekräftigen Fehler zu erzeugen.

3.9 Tests und Qualitätssicherung

Die Anwendung wurde umfassend durch automatisierte Tests abgesichert. Dabei werden sowohl die fachliche Logik als auch die einzelnen technischen Schichten der Anwendung getestet. Die Tests sind entsprechend der Programmstruktur in die Bereiche `domain`, `ocr`, `persistence` und `web` aufgeteilt.

Für die Domainlogik existieren Tests für die Kostenaufteilung, die Verarbeitung von Zahlungen und die statistischen Auswertungen. Dabei werden sowohl typische als auch fehlerhafte Eingaben berücksichtigt. Beispielsweise werden unterschiedliche Artikelzuordnungen, ungerade Centbeträge und fehlende Zuordnungen geprüft. Auch die OCR-Komponenten werden separat getestet. Für die Datenmodelle, Bildvorverarbeitung, Qualitätsbewertung, das Parsing und die Tesseract-Schnittstelle existieren eigene Tests. Dadurch können große Teile der OCR-Pipeline unabhängig von einer vollständigen OCR-Verarbeitung getestet werden. Für die OCR-Entwicklung wurden zusätzlich 27 unterschiedliche Kassenbonbilder verwendet, um die Erkennungsqualität während der Entwicklung praktisch zu überprüfen. Diese Bilder sind jedoch nicht Bestandteil der automatisierten Tests, sondern dienten der manuellen Evaluation und Weiterentwicklung der OCR-Pipeline. Auch der Datenbankzugriff wird separat getestet. Für die Webanwendung existieren sowohl Tests einzelner Routen und Funktionen als auch Tests vollständiger Abläufe. Dadurch wird beispielsweise überprüft, ob typische Benutzerinteraktionen von der Eingabe bis zur Speicherung korrekt funktionieren.

Zusätzlich wird Hypothesis für Property-Based Testing eingesetzt. Dabei werden automatisch zahlreiche unterschiedliche Eingaben erzeugt und allgemeine Eigenschaften der Implementierung überprüft. Eine solche Property stellt beispielsweise sicher, dass die Summe der auf Mitglieder verteilten Anteile immer exakt dem ursprünglichen Artikelpreis entspricht.

Die Kombination aus Unit-Tests, Integrationstests und Property-Based Testing ermöglicht somit eine breite Absicherung der Anwendung. Neben einzelnen Funktionen werden auch das Zusammenspiel der Komponenten und typische vollständige Anwendungsabläufe überprüft.

3.10 Design- und Technologieentscheidungen

Bei der Auswahl der verwendeten Technologien wurden sowohl die Anforderungen des Projekts als auch bereits vorhandene Kenntnisse berücksichtigt. Die Entscheidungen wurden daher nicht ausschließlich nach einem objektiven Vergleich aller verfügbaren Alternativen getroffen.

Als Datenbank wurde PostgreSQL gewählt. Mit PostgreSQL hatte bereits bei einem vorherigen Softwareprojekt innerhalb eines anderen Moduls praktische Erfahrung. Insbesondere der Einsatz von PostgreSQL in Verbindung mit Docker war dadurch bereits bekannt. Die vorhandenen Kenntnisse reduzierten den Einarbeitungsaufwand und ermöglichten es, den Schwerpunkt der Arbeit auf die eigentliche Anwendung und insbesondere die OCR-Pipeline zu legen. Eine andere relationale Datenbank wäre grundsätzlich ebenfalls geeignet gewesen; die Entscheidung für PostgreSQL war daher zumindest teilweise durch die vorhandene Erfahrung begründet.

Für die Gestaltung der Weboberfläche wurde Bootstrap verwendet. Auch diese Entscheidung beruhte unter anderem auf bereits vorhandenen Erfahrungen aus dem Programmierpraktikum. Bootstrap ermöglichte eine konsistente und responsive Gestaltung der Oberfläche, ohne einen großen Teil der Entwicklungszeit in eine eigene CSS-Struktur investieren zu müssen. Eine vollständig eigene Gestaltung wäre möglich gewesen, hätte für die eigentliche Funktionalität des Projekts jedoch keinen wesentlichen zusätzlichen Nutzen geboten.

Auch die Strukturierung des Python-Projekts orientiert sich an der Paketstruktur, die bereits aus dem Programmierpraktikum bekannt war. Die Trennung in `domain`, `persistence`, `ocr` und `web` wurde gewählt, um fachliche Logik, Datenbankzugriff, OCR-Verarbeitung und Weboberfläche voneinander zu trennen. Dadurch können einzelne Komponenten unabhängig voneinander getestet und weiterentwickelt werden. Gleichzeitig wurde auf eine unnötig komplexe Architektur verzichtet, da der Umfang der Anwendung eine wesentlich umfangreichere Architektur nicht erforderlich machte.

Für die OCR wurde zunächst Tesseract eingesetzt und im Verlauf der Entwicklung mit EasyOCR und PaddleOCR verglichen. Die finale Entscheidung für Tesseract basiert daher im Gegensatz zu einigen anderen Technologieentscheidungen nicht primär auf vorhandenen Vorkenntnissen, sondern auf praktischen Tests innerhalb des Projekts. Die Gründe hierfür wurden bereits im Abschnitt zur Entwicklung und Evaluation der OCR-Lösung beschrieben.

Insgesamt wurden somit nicht alle Technologien anhand eines vollständigen objektiven Vergleichs ausgewählt. Insbesondere bei PostgreSQL, Bootstrap und der Paketstruktur spielten vorhandene Erfahrungen und die dadurch geringere Einarbeitungszeit eine wichtige Rolle. Diese bewusste Pragmatisierung ermöglichte es, die verfügbare Entwicklungszeit stärker auf die projektspezifischen Herausforderungen zu konzentrieren.

4 Ergebnisse

4.1 Funktionsnachweis

Die Anwendung bietet nach der Implementierung einen vollständigen Ablauf von der Bonerfassung bis zur Kostenverteilung. Ein typischer Ablauf ist:

1. Ein Benutzer lädt ein Foto eines Kassenbons hoch.
2. Die OCR-Pipeline verarbeitet das Bild.
3. Händler, Datum, Gesamtbetrag und Artikel werden aus den OCR-Kandidaten ermittelt.
4. Die erkannten Daten werden im Formular dargestellt.
5. Der Benutzer kann erkannte Informationen korrigieren.
6. Artikel werden einem oder mehreren Mitgliedern zugeordnet.
7. Der Kassenbon wird gespeichert.
8. Die Anwendung berechnet daraus die individuellen Anteile.
9. Die daraus entstehenden offenen Beträge erscheinen in der globalen Übersicht.
10. Eine bereits erfolgte Zahlung kann als `Payment` erfasst werden.

4.2 Beispiel-Durchlauf

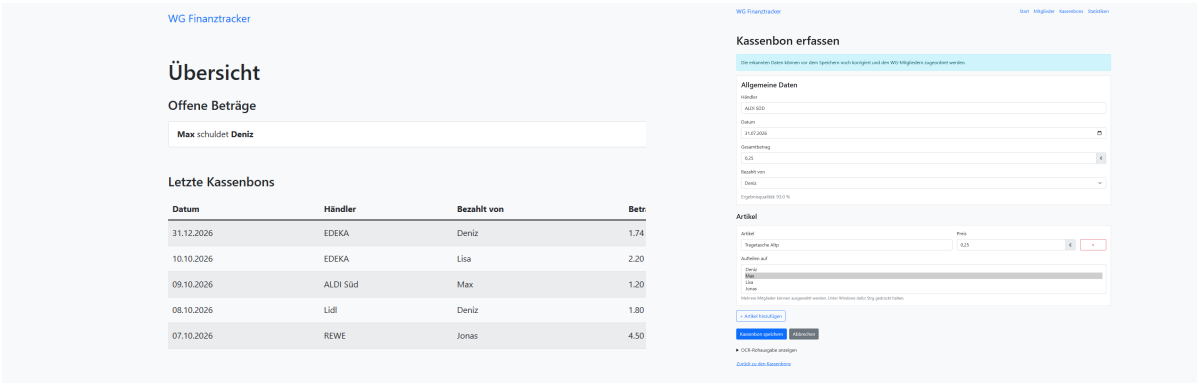
Ein Beispielkassenbon wird zunächst als Bild hochgeladen. Die OCR-Pipeline erzeugt mehrere Kandidaten aus unterschiedlichen Bildvarianten und Tesseract-Konfigurationen. Ein Kandidat

kann beispielsweise einen Händler und einen Gesamtbetrag korrekt erkennen, während ein anderer Kandidat bessere Artikelinformationen liefert. Der kombinierte Ansatz versucht daher, die jeweils belastbareren Informationen der Kandidaten zu verwenden. Nach der OCR-Erkennung können die Ergebnisse im Webformular kontrolliert werden. Falsch erkannte oder unvollständige Daten können vor dem Speichern manuell korrigiert werden. Nach dem Speichern wird aus den Artikelzuordnungen die Kostenaufteilung berechnet. Wird ein Artikel von zwei Personen geteilt, wird sein Preis in Cent auf beide Personen verteilt. Der daraus resultierende Betrag wird anschließend in die globale Schuldübersicht einbezogen.

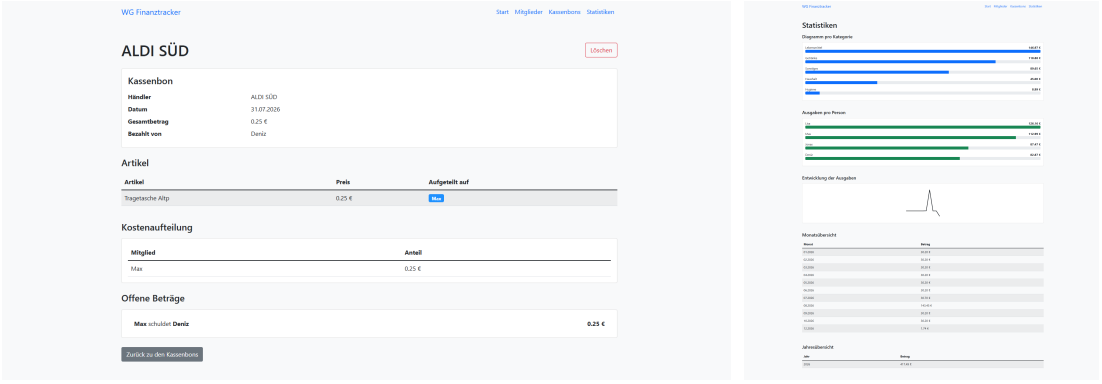
4.3 Ergebnisse visualisieren

Die wichtigsten Funktionen der Anwendung sind in Abbildung 2 anhand von vier Ansichten dargestellt. Die Startseite zeigt die globale Übersicht der aktuell offenen Beträge zwischen den Mitgliedern. Die OCR-Ansicht zeigt den Ablauf nach dem Hochladen eines Kassenbons und die Möglichkeit, erkannte Daten vor dem Speichern zu überprüfen und zu korrigieren. Die Detailansicht eines Kassenbons stellt die enthaltenen Artikel sowie deren Zuordnung zu den Mitgliedern dar. Die Statistikseite zeigt die Auswertung der erfassten Ausgaben nach verschiedenen Kategorien und Zeiträumen.

Die Abbildungen dienen damit sowohl als Funktionsnachweis als auch zur Veranschaulichung des vorgesehenen Benutzerablaufs.



(a) Startseite mit globaler Schuldenübersicht (b) OCR-Ergebnis und Bearbeitungsformular



(c) Kassenbon-Detailseite (d) Statistikseite

Abbildung 2: Ansichten des WG-Finanztrackers

5 Diskussion und Fazit

5.1 Zielerreichung

Die Ziele des Projekts wurden umgesetzt. Die Anwendung ermöglicht die Verwaltung von Mitgliedern und Kassenbons, die Zuordnung von Artikeln zu Mitgliedern sowie die Berechnung von individuellen Anteilen und offenen Beträgen. Die OCR-Funktion wurde ebenfalls implementiert und um mehrere Bildvarianten, unterschiedliche Tesseract-Konfigurationen und eine Bewertung der Ergebnisse erweitert. Dadurch ist die Lösung nicht auf eine einzige Kassenbonstruktur beschränkt. Gleichzeitig bleibt die OCR der technisch unsicherste Teil des Projekts. Die Qualität hängt stark von Bildqualität, Perspektive, Schriftbild und Kassenbonlayout ab. Die Möglichkeit, erkannte Daten manuell zu korrigieren, ist deshalb ein wesentlicher Bestandteil der Anwendung.

5.2 Herausforderungen

Die größte technische Herausforderung war die OCR-Verarbeitung. Kassenbons enthalten neben den eigentlichen Artikeln zahlreiche weitere Informationen wie Steuerzeilen, Rabattinformationen, Zahlungsdaten, Logos und Transaktionsdaten. Diese Informationen können von Tesseract teilweise ähnlich wie Artikelzeilen erkannt werden. Zu Beginn wurde eine einfache Tesseract-basierte Lösung verwendet. Erste Tests zeigten jedoch, dass die Erkennung je nach Kassenbon und Bildqualität stark variierte. Deshalb wurden zunächst alternative OCR-Bibliotheken untersucht und praktisch getestet. Nach Tesseract wurde EasyOCR eingesetzt und anschließend auch PaddleOCR erprobt. Beide Ansätze lieferten für die betrachteten Kassenbons jedoch nicht die erwartete Verbesserung. Daraufhin wurde Tesseract erneut verwendet, die ursprüngliche Lösung aber grundlegend erweitert. Statt nur eine einzelne OCR-Auswertung durchzuführen, werden nun mehrere Vorverarbeitungsvarianten und unterschiedliche Tesseract-Page-Segmentation-Modi verwendet. Die daraus entstehenden OCR-Ergebnisse werden als Kandidaten behandelt und anhand mehrerer Kriterien bewertet. Dabei wird nicht ausschließlich die von Tesseract gelieferte Confidence betrachtet, da eine hohe Zeichensicherheit nicht automatisch eine fachlich korrekte Erkennung bedeutet. Eine weitere Herausforderung bestand daher darin, aus mehreren OCR-Ergebnissen ein plausibles Gesamtergebnis zu bestimmen. Hierfür wurde eine eigene Qualitätsbewertung entwickelt, die unter anderem die Erkennung von Händler, Datum, Gesamtbetrag und Artikeln sowie die Plausibilität der erkannten Preise berücksichtigt. Im Verlauf der Entwicklung zeigte sich, dass die Wahl der OCR-Bibliothek allein nicht ausreicht, um unterschiedliche Kassenbonlayouts zuverlässig zu verarbeiten. Die Vorverarbeitung des Bildes, die Wahl der OCR-Konfiguration und insbesondere die nachträgliche Verarbeitung und Plausibilisierung der erkannten Daten haben einen wesentlichen Einfluss auf das Ergebnis. Deshalb wurde die finale Lösung bewusst nicht auf einen einzelnen Händler zugeschnitten. Im Bereich der Webanwendung lag eine weitere Herausforderung darin, fehlerhafte Eingaben robust zu behandeln. Gerade bei OCR-Ergebnissen muss davon ausgegangen werden, dass einzelne Werte fehlen oder falsch erkannt werden. Die erkannten Daten können deshalb vor dem Speichern manuell kontrolliert und korrigiert werden.

5.3 Kritische Reflexion

Die Verwendung von heuristischen Regeln im OCR-Parser bietet den Vorteil, dass die Implementierung ohne ein eigenes Machine-Learning-Modell auskommt und nachvollziehbar bleibt. Gleichzeitig sind solche Regeln naturgemäß empfindlich gegenüber bisher unbekannten Bonlayouts. Die mehrstufige Kandidatenauswahl verbessert die Flexibilität gegenüber einer einzelnen OCR-Konfiguration, führt aber auch zu zusätzlicher Komplexität. Insbesondere die Gewichtung der Kandidaten ist teilweise heuristisch gewählt und könnte mit einem größeren Datensatz systematischer evaluiert werden. Auch die Entwicklung mit KI-Unterstützung hatte Vor- und Nachteile. KI-Unterstützung war insbesondere bei Standardcode, Tests und der Analyse von

Fehlermeldungen hilfreich. Bei projektspezifischen Entscheidungen mussten Vorschläge jedoch wiederholt angepasst oder verworfen werden. Die tatsächliche Verantwortung für die Architektur und die fachliche Korrektheit blieb daher bei der manuellen Überprüfung.

5.4 Abweichungen vom ursprünglichen Plan

Die Implementierung wurde gegenüber der ursprünglichen Minimalidee erweitert. Neben der grundlegenden Bonverwaltung wurden unter anderem eine globale Schuldenübersicht, Payments, statistische Auswertungen und eine umfangreichere OCR-Pipeline umgesetzt. Die OCR erwies sich als deutlich aufwendiger als zunächst erwartet. Der Schwerpunkt verschob sich deshalb teilweise von einer einfachen Tesseract-Anbindung hin zu Vorverarbeitung, Kandidatenauswahl, Parsing und Plausibilitätsprüfung.

6 Ausblick

Ein möglicher nächster Schritt wäre eine systematische Evaluation der OCR-Pipeline mit einer größeren Sammlung unterschiedlicher Kassenbons. Dabei könnten Händler, Bildqualität und Layout separat betrachtet und die verschiedenen Vorverarbeitungsvarianten quantitativ verglichen werden. Außerdem könnte die Erkennung komplexerer Bonstrukturen erweitert werden. Denkbar wären beispielsweise robustere Erkennung von Mengenangaben, weitere Layouttypen und verbesserte Zuordnung von Rabatten zu den jeweils betroffenen Artikeln. Im Bereich der Zahlungen wäre langfristig eine explizite Zuordnung einer Zahlung zu einer konkreten Schuld oder einem konkreten Kassenbon denkbar. Dadurch könnten auch einzelne Kassenbons detailliert anzeigen, welcher Teil bereits beglichen wurde.

7 Formale Qualitätssicherung

Die formalen Anforderungen an den Quelltext wurden während der Entwicklung fortlaufend überprüft. Das Projekt verwendet eine `pyproject.toml`, in der die benötigten Abhängigkeiten verwaltet werden. Zusätzlich wurden Fehlerbehandlung und Validierung an den relevanten Eingabepunkten implementiert. Die Versionshistorie wird in einem Git-Repository geführt. Für die Qualitätssicherung wird `pytest` für automatisierte Tests und `Hypothesis` für Property-Based Testing eingesetzt. `mypy` überprüft die statische Typisierung des Quelltexts, während `Ruff` zur Prüfung und Formatierung verwendet wird. Mit `Interrogate` wird außerdem die Dokumentation des Quelltexts überprüft.

8 Design by Contract und verwandte Techniken

Ein formaler Design-by-Contract-Ansatz mit expliziten Pre- und Postconditions wurde nicht verwendet. Es wurden jedoch verwandte Techniken zur Validierung von Vorbedingungen eingesetzt. So werden beispielsweise ungültige Preise, fehlende Artikelzuordnungen und unvollständige Eingaben vor der weiteren Verarbeitung erkannt und gezielt behandelt.

9 Fazit

Das Projekt zeigt, dass sich die Erfassung und Verteilung gemeinschaftlicher Einkaufskosten mit einer Kombination aus Webanwendung, relationaler Datenbank und OCR automatisieren lässt. Besonders die OCR-Verarbeitung erwies sich als anspruchsvoller Bestandteil des Projekts. Die Kombination mehrerer Bildvarianten, Tesseract-Konfigurationen, Parserregeln und einer Qualitätsbewertung ermöglicht eine flexiblere Verarbeitung als eine einfache Einzelabfrage an Tesseract. Gleichzeitig bleibt die manuelle Kontrolle ein wichtiger Bestandteil des Systems. Die

Anwendung nutzt OCR zur Reduzierung des Eingabeaufwands, behandelt die Erkennung aber nicht als vollständig fehlerfreie Datenquelle. Insgesamt wurde die geplante Funktionalität umgesetzt und über die grundlegenden Anforderungen hinaus um eine globale Schuldenübersicht, Payments und statistische Auswertungen ergänzt.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und die verwendeten Hilfsmittel und Quellen vollständig angegeben habe. Die im Rahmen der Arbeit eingesetzte KI-Unterstützung ist im Bericht entsprechend dokumentiert. Übernommene Inhalte wurden überprüft und an den tatsächlichen Projektstand angepasst.

A Ausgewählter Code-Snippet

A.1 Bewertung von OCR-Ergebnissen

Da die von Tesseract gelieferte Confidence allein keine Aussage über die fachliche Qualität des erkannten Kassenbons ermöglicht, wird zusätzlich eine eigene Qualitätsbewertung verwendet. Dabei werden unter anderem erkannter Händler, Datum, Gesamtbetrag und Artikel berücksichtigt. Die Tesseract-Confidence fließt nur mit einem geringen Gewicht in die Gesamtbewertung ein.

```
def score_receipt(
receipt: OcrReceipt,
) -> float:
score = 0.0

if receipt.store is not None:
    score += 0.10

if receipt.purchase_date is not None:
    score += 0.15

if receipt.price_cents is not None and receipt.price_cents > 0:
    score += 0.25

score += score_items(receipt) * 0.45

if receipt.confidence is not None:
    confidence = min(
        max(receipt.confidence, 0.0),
        1.0,
    )
    score += confidence * 0.05

return min(score, 1.0)
```